

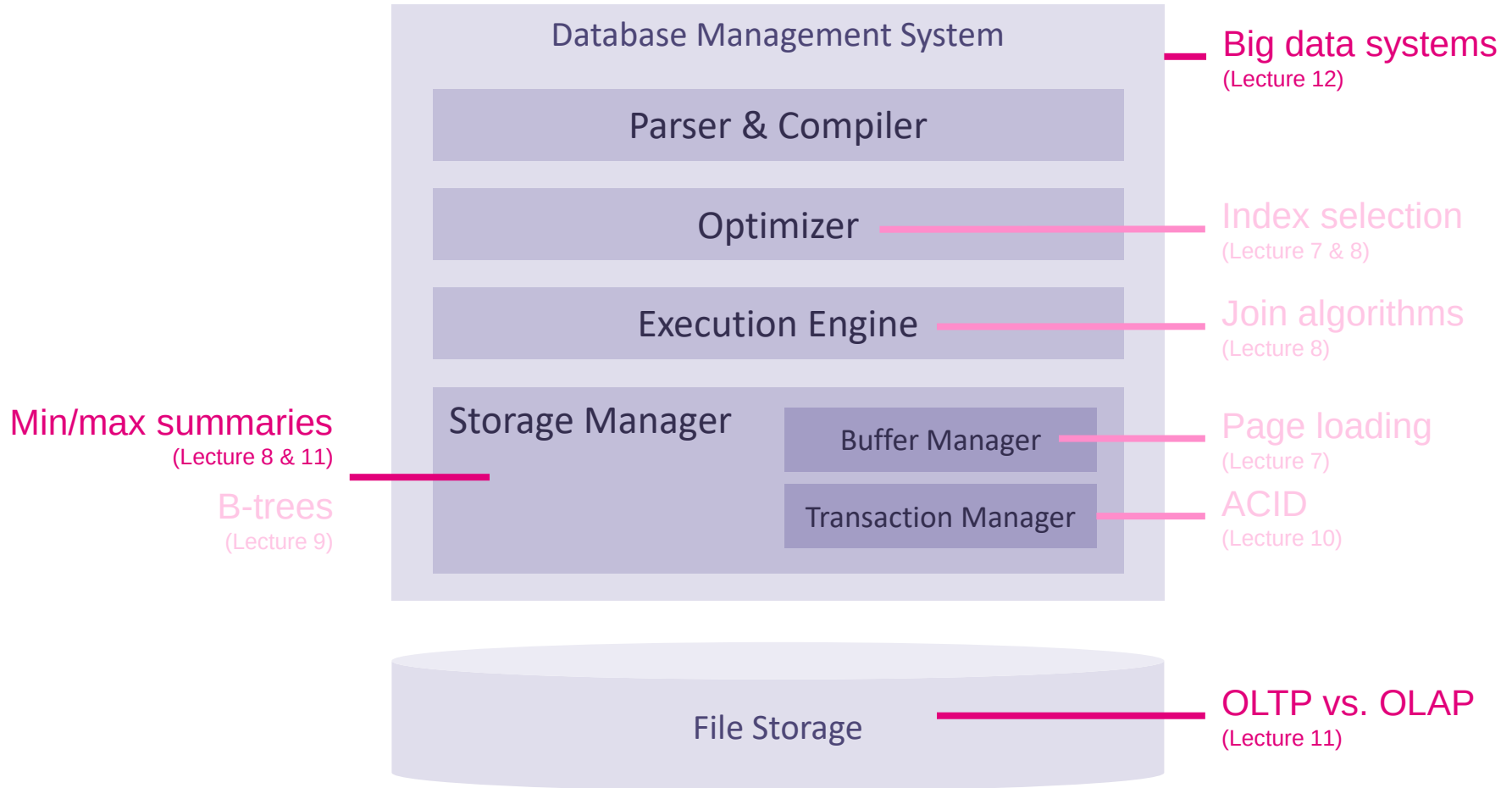


Introduction to Database Systems
Spring 2025, Lecture 11:
OLTP vs. OLAP
Storage Models: NSM vs. DSM vs. PAX

Intended Learning Outcomes

1. Suggest a database design in the ER model and convert to a relational database schema in a suitable normal form.
2. Write SQL queries, involving multiple relations, compound conditions, grouping, aggregation, and subqueries.
3. Use relational DBMSs from a conventional programming language in a secure manner.
4. Analyze/predict/improve query processing efficiency of the designed database using indices.
5. Reflect upon the evolution of the hardware and storage hierarchy and its impact on data management system design.
6. Discuss the pros and cons of different classes of data systems for modern analytics and data science applications.

Components of a Database System



Agenda

OLTP vs. OLAP

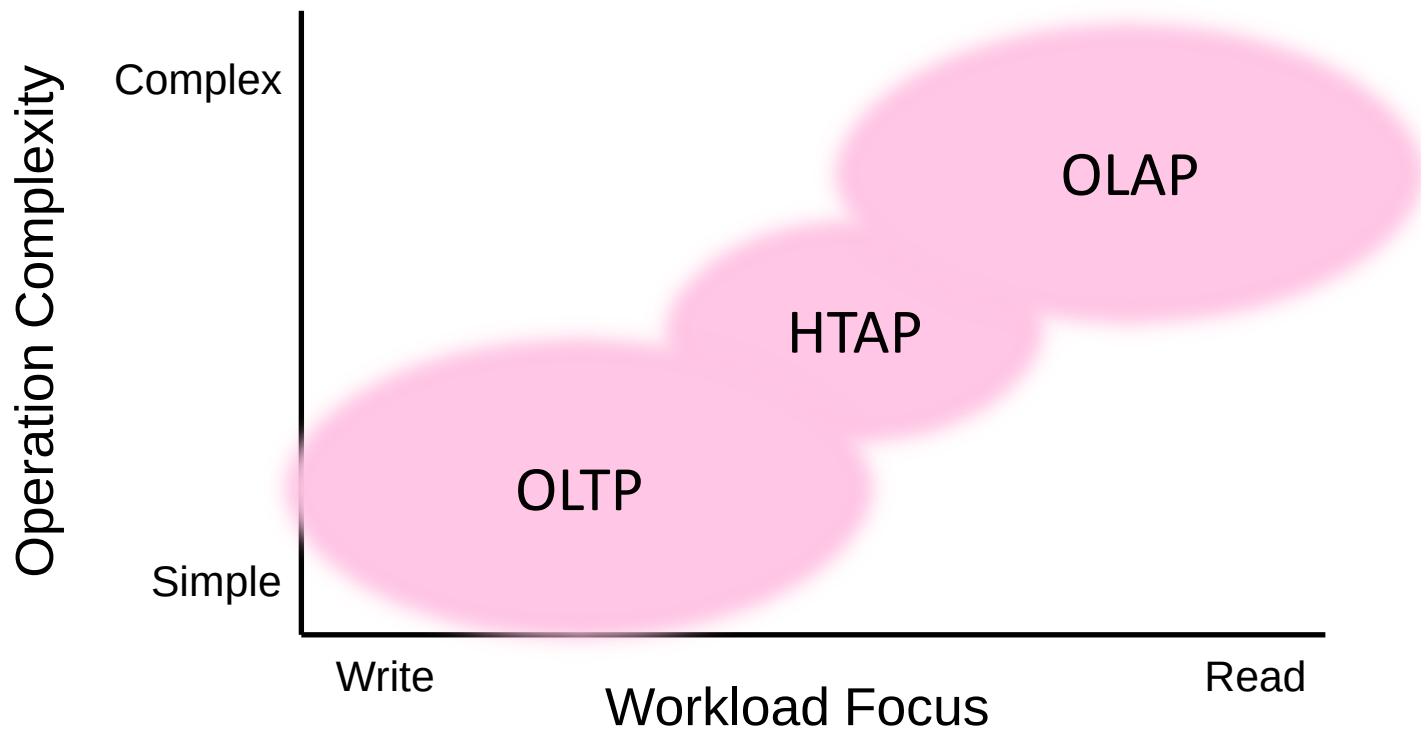
Storage Models

- N-ary Storage Model (NSM)
- Decomposition Storage Model (DSM)
- Partition Attributes Across (PAX)

Snowflake Data Management

- Data Insertion
- Query Optimization via Pruning

Database Workloads



Database Workloads

“Online” = system gives answer immediately
(1980s definition of “online” vs. scheduled jobs on mainframe)

OLTP: Online Transaction Processing

→ Fast operations that only read/update small amount of data each time

OLAP: Online Analytical Processing

→ Complex queries that read a lot of data

HTAP: Hybrid Transaction and Analytical Processing

→ OLTP and OLAP together in the same database

Online Transaction Processing

- Focus on real-time data processing
- High volume of short transactions
- Typically involves frequent insert, update, and delete operations

→ Processes small amounts of data
→ Build applications, e.g. Analog App

Example Queries

```
-- price of a tea
select *
from product
where productname = 'Tea';
```

```
-- price of 'fancy' product
-- is unknown
insert into product
  values('Fancy', null);
```

```
-- update user Donna
update user
set email = 'donna@itu.dk',
    password = '***'
where username = 'Donna';
```

Online Analytical Processing

- Focus on querying and reporting
- Complex queries on large datasets
- Typically involves data aggregation and multi-dimensional analysis

→ Processes large amounts of data

→ Data science, ML, visualizations

Example Queries

```
-- how many products did  
-- each customer buy  
select username,  
        count(*) as count  
from purchase  
group by username  
order by count desc;
```

```
-- cumulative sum of sales  
-- per user over time  
select *,  
        sum(price) over  
        (partition by username  
         order by purchasetime)  
        as cum_sales  
from sales  
order by purchasetime;
```

Exercise

Which workload, OLTP or OLAP, do queries A and B belong to?

Purchase (Id, PurchaseTime, ProductName, UserName)

Id	PurchaseTime	ProductName	UserName
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Small	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin

Query A

```
select productname,  
       count(*) as quantity_sold  
from purchase  
where purchasetime > '2025-02-01'  
and purchasetime < '2025-03-01'  
group by productname;
```

OLAP

Query B

```
update purchase  
set productname = 'Tea'  
where id = 2  
and username = 'Martin';
```

OLTP

Agenda

OLTP vs. OLAP

Storage Models

- N-ary Storage Model (NSM)
- Decomposition Storage Model (DSM)
- Partition Attributes Across (PAX)

Snowflake Data Management

- Data Insertion
- Query Optimization via Pruning

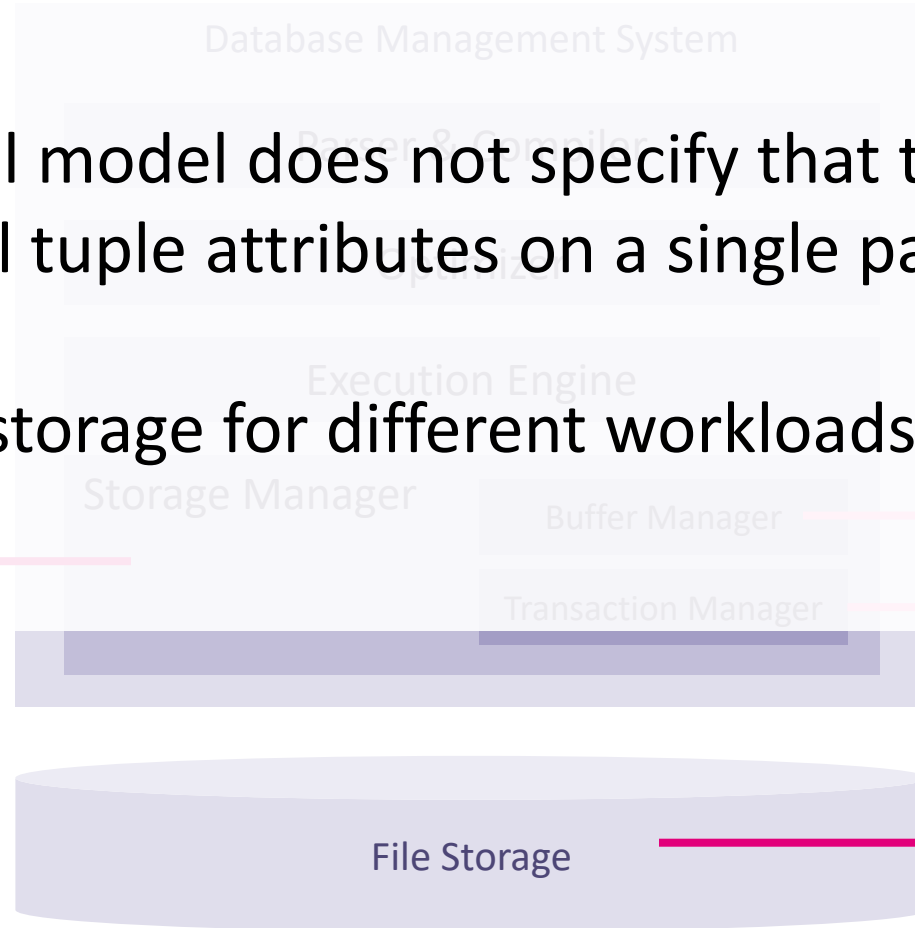
Observation

The relational model does not specify that the DBMS must store all tuple attributes on a single page / file

→ Optimize storage for different workloads

Min/max summaries
(Lecture 8 & 11)

B-trees
(Lecture 9)



Page loading
(Lecture 7)

ACID
(Lecture 10)

OLTP vs. OLAP
(Lecture 11)

Data in Files (see also Lecture 8)

Store data in files by tuples, one tuple after the next

- Header contains metadata like number of tuples, start byte of first tuple, checksums, and more
- Tuples can be of different lengths (for simplicity all same length below)

Purchase (Id, PurchaseTime, ProductName, UserName)

Id	PurchaseTime	ProductName	UserName
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Small	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin



Purchase.data

Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Small	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin

N-ary Storage Model (NSM)

DBMS's **storage model** defines how it organizes tuples on disk and in memory

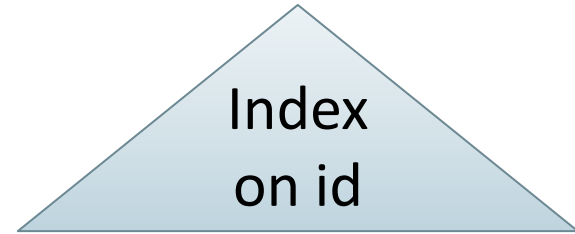
N-ary Storage Model (NSM)

- Store all attributes of tuple in a single file, tuple by tuple
- Known as “row store”

→ Ideal for OLTP workloads where queries are more likely to access individual entities and execute write-heavy workloads

NSM vs. OLTP Example

```
update purchase
set productname = 'Tea'
where id = 2
and username = 'Martin';
```



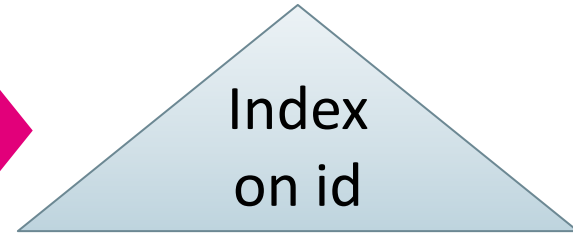
Purchase.data

Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Tea	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin

- Finding the right tuple to update is fast using the index
 - Comparing username and updating productname is fast because we've loaded the full tuple into memory
- 1 index access, 1 tuple access

NSM vs. OLTP Example

```
insert into purchase  
values (6, '2025-04-09  
11:03'::timestamp, 'Small', 'Martin');
```



Purchase.data

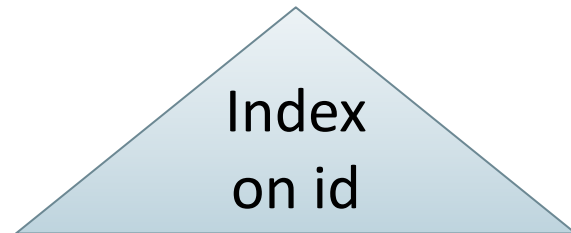
Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Tea	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin
6	Apr 09, 11:03	Small	Martin

- Inserting fast: find a free slot in a file and write the tuple to the end of the file

→ 1 index update, 1 file write

NSM vs. OLAP – Exercise

```
select productname,  
       count(*) as quantity_sold  
from purchase  
where purchasetime > '2025-02-01'  
and purchasetime < '2025-03-01'  
group by productname;
```



Purchase.data

Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Tea	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin
6	Apr 09, 11:03	Small	Martin

- Is NSM a good choice for OLAP queries?
- Can you think of some drawbacks of NSM?

NSM vs. OLAP

```
select productname,  
       count(*) as quantity_sold  
from purchase  
where purchasetime > '2025-02-01'  
and purchasetime < '2025-03-01'  
group by productname;
```

- OLAP query reads all tuples, loading all attributes
- 6 tuple accesses
- > 50% attributes uselessly loaded into memory

Purchase.data

Header			
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Tea	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin
6	Apr 09, 11:03	Small	Martin

Useless data

Advantages

- Fast inserts, updates, and deletes
- Good memory locality for queries that output the entire tuple
- Great for OLTP workloads

Disadvantages

- Not good for scanning large portions of the table and/or a subset of the attributes
- Terrible memory locality for OLAP access patterns
- Not ideal for compression because of multiple value domains within a single file

Decomposition Storage Model (DSM)

Decomposition storage model **decomposes** tuples

- Attributes are stored column by column
 - Known as “column store”
- Ideal for OLAP workloads where read-only queries perform large scans over a subset of the table's attributes
- DBMS is responsible for combining/splitting a tuple's attributes when reading/writing

Purchase (Id, PurchaseTime, ProductName, UserName)

Id	PurchaseTime	ProductName	UserName
1	Feb 11, 9:55	Tea	Martin
2	Feb 12, 10:03	Small	Martin
3	Feb 12, 10:05	Small	Omar
4	Feb 12, 10:06	Large	Omar
5	Feb 19, 9:00	Small	Martin



Purchase.dsm



DSM vs. OLAP Example

```
select productname,  
       count(*) as quantity_sold  
from purchase  
where purchasetime > '2025-02-01'  
and purchasetime < '2025-03-01'  
group by productname;
```

Filter only reads
purchasetime
attributes

Group by only reads
productname attributes

Purchase.dsm

Header					
1	2	3	4	5	
Feb 11, 9:55			Feb 12, 10:03		
Feb 12, 10:05			Feb 12, 10:06		
Feb 19, 9:00					
Tea	Small	Small	Large		
Small					
Martin	Martin		Omar		
Omar	Martin				

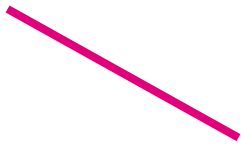
DSM vs. OLTP – Exercise

Drawbacks of using DSM for OLTP workloads

select *  **from** purchase
where id = 2
 and username = 'Martin';

Reading from multiple pages
or files + tuple reconstruction
costs

insert into purchase
values (6, '2025-04-09 11:03'::timestamp,
 'Small', 'Martin');

 Tuple deconstruction
costs + writing to
multiple pages or files

Purchase.dsm

Header					
1	2	3	4	5	6
Feb 11, 9:55			Feb 12, 10:03		
Feb 12, 10:05			Feb 12, 10:06		
Feb 19, 9:00			Apr 9, 11:03		
Tea	Small	Small	Large		
Small	Small				
Martin	Martin	Omar			
Omar	Martin	Martin			

Advantages

- Reduces wasted I/O when scanning large amounts of data
 - Better cache locality when working on a single attribute
 - Better data compression
- Great for OLAP workloads!

Disadvantages

- Slow for point queries, insert, updates, deletes because of tuple construction and deconstruction, and reading/writing in multiple locations

Systems in the Wild

Row Store Databases



Column Store Databases



Systems in the Wild

Big vendors offer different database products with row stores, column stores, and hybrid storage models



Newer OLAP systems and file formats with hybrid storage models



Apache Parquet is an open-source file format, not a full database system

Hybrid Storage Model

Of course, it's not as simple as just NSM vs. DSM...

People industry and academia thought about the spectrum in between, e.g. in this research paper from 2001:

Weaving Relations for Cache Performance

Anastassia Ailamaki[‡]
Carnegie Mellon University
natassa@cs.cmu.edu

David J. DeWitt
Univ. of Wisconsin-Madison
dewitt@cs.wisc.edu

Mark D. Hill
Univ. of Wisconsin-Madison
markhill@cs.wisc.edu

Marios Skounakis
Univ. of Wisconsin-Madison
marios@cs.wisc.edu

Abstract

Relational database systems have traditionally optimized for I/O performance and organized records sequentially on disk pages using the N-ary Storage Model (NSM) (a.k.a., slotted pages). Recent research, however, indicates that cache utilization and performance is becoming increasingly important on modern platforms. In this paper, we first demonstrate that in-page data placement is the key to high cache performance and that NSM exhibits low cache utilization on modern platforms. Next, we propose a new data organization model called PAX (Partition Attributes Across), that significantly improves cache performance by grouping together all values of each attribute within each page. Because PAX only affects layout inside the pages, it

tremendous additional time to join the participating sub-relations together. Except for Sybase-IQ [33], today's relational DBMSs use NSM for general-purpose data placement [20][29][32].

Recent research has demonstrated that modern database workloads, such as decision support systems and spatial applications, are often bound by delays related to the processor and the memory subsystem rather than I/O [20][5][26]. When running commercial database systems on a modern processor, data requests that miss in the cache hierarchy (i.e., requests for data that are not found in any of the caches and are transferred from main memory) are a key memory bottleneck [1]. In addition, only a fraction of

Partition Attributes Across (PAX)

Research Paper:

DSM good for I/O, but poor tuple reconstruction cost

NSM poor I/O, but no tuple reconstruction overhead

→ Combine both, call it PAX

typically not the same size. Loading the cache with useless data (a) wastes bandwidth, (b) pollutes the cache, and (c) possibly forces replacement of information that may be needed in the future, incurring even more delays. The challenge is to repair NSM's cache behavior without compromising its advantages over DSM.

This paper introduces and evaluates **Partition Attributes Across (PAX)**, a new layout for data records that combines the best of the two worlds and exhibits performance superior to both placement schemes by eliminating unnecessary accesses to main memory. For a given relation, PAX stores the same data on each page as NSM. *Within* each page, however, PAX groups all the values of a particular attribute together on a minipage. During a

Remember row groups from Lecture 8?

Product.data

Header	
Small, 85	Tea, 100
Large, 100	Filter, 110
Fancy, 125	Espresso, 35
Cortado, 35	Mocha, 45
Jasmin, 30	Cola, 25

Row group 1

Row group 2

Row group 3

NSM tuple storage



Product.pax

Header	
Small, Tea, Large, Filter	
85, 100, 100, 110	
Fancy, Espresso, Cortado	
Mocha	125, 35, 30, 25
Jasmin, Cola	30, 25

DSM tuple storage
per row group

PAX Adoption (Examples)



Product.duckdb

Header	
Small, Tea, Large, Filter	
85, 100, 100, 110	
Fancy, Espresso, Cortado	
Mocha	125, 35, 30, 25
Jasmin, Cola	30, 25

1 file split into row groups using DSM tuple storage



Product1.data

Header
Small, Tea, Large, Filter
85, 100, 100, 110

Product2.data

Header
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25

Product3.data

Header
Jasmin, Cola 30, 25

1 or more files with 1 row group each using DSM tuple storage



Product1.parquet

Header
Small, Tea, Large, Filter
85, 100, 100, 110
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25
Jasmin, Cola 30, 25

Product2.parquet

Header
Small, Tea, Large, Filter
85, 100, 100, 110
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25
Jasmin, Cola 30, 25

Product3.parquet

Header
Small, Tea, Large, Filter
85, 100, 100, 110
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25
Jasmin, Cola 30, 25

Product4.parquet

Header
Small, Tea, Large, Filter
85, 100, 100, 110
Fancy, Espresso, Cortado
Mocha 125, 35, 30, 25
Jasmin, Cola 30, 25

1 or more files with 1 or more row groups each using DSM tuple storage

PAX Summary

Advantages

- Better I/O than NSM
 - Better tuple reconstruction than pure DSM, because attributes are “closer” together, e.g., on the same memory page as proposed in the original paper
 - Many practical benefits, e.g., inserts only affect the last row group = one file write
- PAX used in almost all commercial OLAP database systems

Disadvantages

- Tradeoff: pure columns stores better in special scenarios, e.g., specific benchmarks

Who's the Winner?



OLTP vs. OLAP?

Need both!

NSM vs. DSM / PAX?

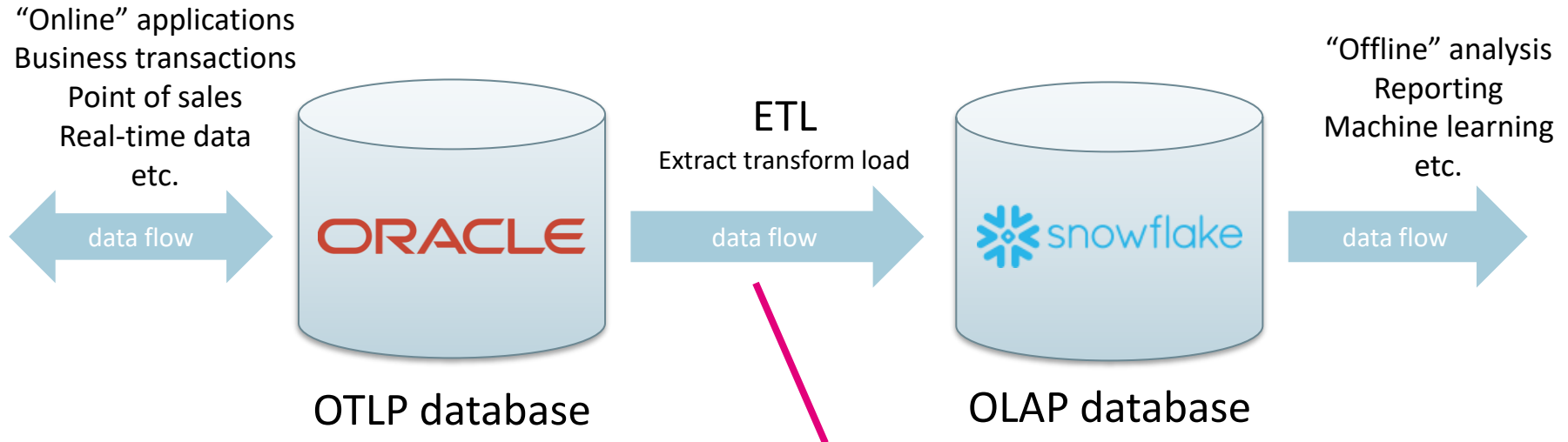
(Row store database vs. column store database)

No system that can do both!

Industry State of the Art

Typical set up of databases in the industry:

- DB for OLTP: running applications and “online” workloads
 - DB for OLAP: doing “offline” analysis, reporting, ML
- because of different storage models, no system that fits all



ETL is a whole market on its own,
e.g. products like dbt and Fivetran

OLTP vs. OLAP

Storage Models

- N-ary Storage Model (NSM)
- Decomposition Storage Model (DSM)
- Partition Attributes Across (PAX)

Snowflake Data Management

- Data Insertion
- Query Optimization via Pruning

Snowflake Data Management

Example of how Snowflake ingests data

users.csv

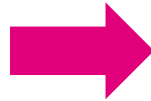
```
ID, Name
1, Pinar
2, Veronika
3, Dovile
4, Zoi
5, Martin
6, Eleni
```

Copy Data into Snowflake

copy into users from @stage;

users.csv

ID	Name
1	Pinar
2	Veronika
3	Dovile
4	Zoi
5	Martin
6	Eleni



Snowflake query engine reads
CSV and produces data files in
PAX format and metadata



users1.data

Header
1, 2, 3
Pinar, Veronika, Dovile

users2.data

Header
4, 5, 6
Zoi, Martin, Eleni

**Metadata
M1 at 2pm**

Added:
user1.data,
users2.data

Deleted:

Min/max:
ID: 1-3, 4-6
Name: D-V, E-Z

Inserting Data Adds Files and Metadata

insert into users values (7, 'Kasper');



users3.data

Header
7
Kasper

**Metadata
M2 at 3pm**

Added: user3.data

Deleted:

Min/max:

ID: 7-7

Name: K-K

→
Snowflake query
engine produces data
file in PAX format and
metadata additional
metadata

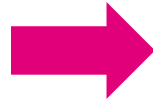
Deleting Data Adds(!) Files and Metadata

delete from users where name = 'Zoi';

Snowflake query engine copies remaining data from user2.data into a new file and produces metadata

users2.data

Header
4, 5, 6
Zoi, Martin, Eleni



users4.data

Header
5, 6
Martin, Eleni



Metadata
M3 at 4pm

Added: user4.data
Deleted: user2.data
Min/max:
ID: 5-6
Name: E-M

Deletion
marked in
metadata

Data and Metadata After 3 DMLs

4 data files, 3 metadata entries

→ All stored in the cloud

users1.data

Header
1, 2, 3
Pinar, Veronika, Dovile

users3.data

Header
7
Kasper

users2.data

Header
4, 5, 6
Zoi, Martin, Eleni

users4.data

Header
5, 6
Martin, Eleni

M1 at 2pm

Added:

user1.data,
users2.data

Deleted:

Min/max:

ID: 1-3, 4-6

Name: D-V, E-Z

M2 at 3pm

Added: user3.data

Deleted:

Min/max:

ID: 7-7

Name: K-K

M3 at 4pm

Added: user4.data

Deleted: user2.data

Min/max:

ID: 5-6

Name: E-M

Query Optimization via Pruning

Pruning based on query predicates

select * from users;

load latest committed metadata: M1, M2, M3

scan set: users1.data, users3.data, users4.data

no pruning (no predicates)

query engine applies no filter

select * from users where ID = 5;

load latest committed metadata: M1, M2, M3

scan set: users1.data, users3.data, users4.data

pruned scan set: users4.data

query engine applies filter ID = 5 to rows in users4.data

M1 at 2pm

Added:

user1.data,
users2.data

Deleted:

Min/max:

ID: 1-3, 4-6

Name: D-V, E-Z

M2 at 3pm

Added: user3.data

Deleted:

Min/max:

ID: 7-7

Name: K-K

M3 at 4pm

Added: user4.data

Deleted: user2.data

Min/max:

ID: 5-6

Name: E-M

Versioning and Time Travel

Assume M3 is not committed yet

select * from users where ID = 5;

load latest committed metadata: M1, M2

scan set: users1.data, users2.data, users3.data

pruned scan set: users2.data

query engine applies filter ID = 5 to rows in user2.data

select * from users at (timestamp => '2pm')
where ID = 5;

load committed metadata before and at 2PM: M1

scan set: users1.data, users2.data

pruned scan set: users2.data

query engine applies filter ID = 5 to rows in users2.data

M1 at 2pm

Added:

user1.data,
users2.data

Deleted:

Min/max:

ID: 1-3, 4-6

Name: D-V, E-Z

M2 at 3pm

Added: user3.data

Deleted:

Min/max:

ID: 7-7

Name: K-K

M3 at 4pm

Added: user4.data

Deleted: user2.data

Min/max:

ID: 5-6

Name: E-M

Problem: Bad Pruning

Predicates on non-clustered columns

select * from users where name = 'Kasper';

load latest committed metadata: M1, M2, M3

scan set: users1.data, users3.data, users4.data

pruned scan set: **users1.data, users3.data, users4.data**

query engine filters rows in all files



Bad pruning leads to bad query runtime

- No pruning was possible → all files must be read
 - Different data layout would help (e.g., sorting by name)
- Something for MSc courses, or find details in Section 3.1:
[Building a Data Management System for the Cloud: Lessons Learned and Future Directions](#), Datenbank-Spektrum 2025

M1 at 2pm

Added:

user1.data,
users2.data

Deleted:

Min/max:

ID: 1-3, 4-6

Name: D-V, E-Z

M2 at 3pm

Added: user3.data

Deleted:

Min/max:

ID: 7-7

Name: K-K

M3 at 4pm

Added: user4.data

Deleted: user2.data

Min/max:

ID: 5-6

Name: E-M

Takeaways

OLTP vs. OLAP

- Different data access patterns
- OLTP: Processes small amounts of data, typically short transactions
- OLAP: Scans large amounts of data, typically long read-only workloads
- HTAP: Mostly a buzzword. ETL processes to go from OLTP → OLAP DBs

Storage Models

- NSM: Row-wise storage
- DSM: Column-wise storage
- PAX: Column-wise per partition (row group)

Reading Material

- [Weaving Relations for Cache Performance](#), VLDB 2001 (optional)
- Section 3.1 in [Building a Data Management System for the Cloud: Lessons Learned and Future Directions](#), Datenbank-Spektrum 2025